

Technical Report

Design and Verification of a 25¢ Moore Vending Machine FSM

Reece M. Dobro

Abstract

The process of designing and testing a Moore state machine for a vending machine using the Verilog hardware description language within AMD Vivado is presented in this report. The FSM was implemented using behavioral modeling to process dime (10¢) and quarter (25¢) inputs and to dispense a 25¢ product. Two simulation scenarios were created to evaluate both ideal and non-ideal input conditions. The results of the simulations were analyzed using waveform outputs and TCL console logs, which confirmed that the state transitions and outputs matched the expected behavior defined by the state transition table. Overall, the simulation results demonstrate that the FSM operates correctly and satisfies the functional requirements specified for the design.

1 Introduction, Background, and Theory

The main objective of this project is to design, program, and test a Moore finite state machine (FSM) for a vending machine in Verilog using behavioral-level modeling. The design implements a system that accepts 10-cent (dime) and 25-cent (quarter) inputs to dispense a 25-cent product. A Moore FSM was chosen for this application as per project specifications. The operation of this FSM involves tracking the total accumulated value across five states, ranging from 0 cents (S0) to over 25 cents (S5). Transitions between these states occur on the rising edge of the clock signal. Additionally, this design is implemented to handle non-standard input sequences and invalid logic states, such as inserting two coins or being in an invalid state, by either maintaining the current state or returning to the idle state S0.

1.1 Inputs and Outputs

The inputs, outputs, and expected behavior were provided by the system technical requirements for this assignment. The module, *vending_25c_moore*, has three inputs and two 1-bit outputs. These inputs and outputs are shown in Table 1.

Signal	Direction	Size	Description
clock	Input	1	Free-running system clock for synchronous state transitions
reset_n	Input	1	Active-low synchronous reset to return the FSM to state S0
DQ	Input	2	2-bit coin input: DQ[1] = Dime (10¢), DQ[0] = Quarter (25¢)
P	Output	1	Product output; asserted when 25¢ or more is accumulated
C	Output	1	Change output; asserted when more than 25¢ is accumulated

Table 1: Module Port Signal and Function Table

As shown by Table 1, the module takes in a 2-bit vector input, *DQ*, which represents the insertion of dimes (bit 1) and quarters (bit 0). For example, $DQ=2'b01$ is equivalent to a single quarter being inserted. It is important to note that two coins cannot be inserted at once (represented by $DQ=2'b11$); if this occurs, it is treated as $DQ=2'b00$ (no coins inserted). Aside from *DQ*, the module has a 1-bit input for a clock signal, *clock*, and a 1-bit input for an active-low synchronous reset signal, *reset_n*, to initialize the state to S0. At the start of any simulation for the program, $reset_n = 1'b0$, in order to set FSM to a known state. The outputs consist of a 1-bit product signal, *P*, and a 1-bit change signal, *C*. As this is a Moore machine, these outputs are dependent on the current state.

1.2 States

The FSM requires five states to track the total amount of money entered, ranging from 0 cents to over 25 cents. These states and their corresponding Moore outputs are detailed in Table 2. The columns P and C represent the 1-bit product and change outputs mentioned earlier. The minimum size for the state register, k , is determined by the number of unique states, n , according to the formula $k = \lceil \log_2(n) \rceil$. Since there are $n = 5$ distinct states required, the calculation becomes $k = \lceil \log_2(5) \rceil = 3$. Therefore, a 3-bit state register is necessary to hold the *current_state* and *next_state* variables.

As a result of this 3-bit encoding, the state variables can represent up to eight unique values, leaving three unused state assignments. Per the project specifications, any unused state assignments are set to transition to S_0 with all outputs driven to zero to ensure predictable hardware behavior

State	Total Value	Encoding	P (Product)	C (Change)
S0	0¢	000	0	0
S1	10¢	001	0	0
S2	20¢	010	0	0
S3	25¢	011	1	0
S4	> 25	100	1	1

Table 2: State Encoding and Moore Output Table

Because this is a Moore FSM, proper state transitions are required for correct operation. A state transition table is shown by Table 3. Per the project specifications, if a "00" or "11" input is received while in a state where a product isn't received (S_0 – S_2), the machine remains in its current state. If any input is received while in a state where a product is received (S_3 or S_4), the machine transitions back to S_0 to begin a new transaction.

Current State	DQ = 00/11 (Idle/Invalid)	DQ = 10 (Dime)	DQ = 01 (Quarter)
S0 (0¢)	S0	S1	S3
S1 (10¢)	S1	S2	S4
S2 (20¢)	S2	S4	S4
S3 (25¢)	S0	S0	S0
S4 (> 25)	S0	S0	S0

Table 3: State Transition Table

1.3 State Diagram

Figure 1 shows the state diagram created for vending machine FSM. As the outputs are defined by the states they are in, this diagram is for a Moore state machine, as per project specifications. The state diagram serves as a visual synthesis of the inputs, outputs, and state transitions detailed in the previous sections. The *reset_n* pointing towards state S_0 signifies that when *reset_n* is driven low, the state machine's next state becomes S_0 . The bottom arrow from S_3 and S_4 , the product states, show how the next state of the FSM becomes S_0 after reaching S_3 or S_4 , without regard to the input. This behavior was required by the project specifications.

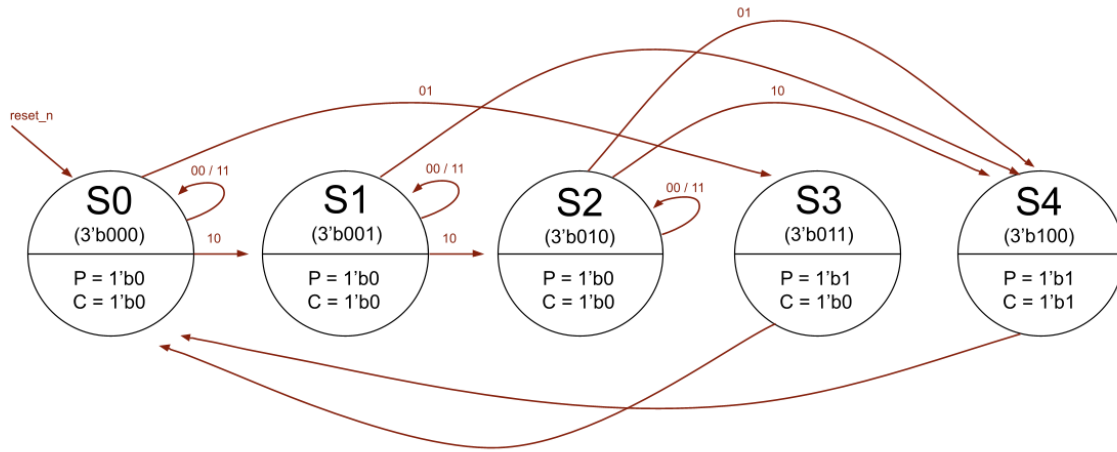


Figure 1: State Diagram for the *vending_25c_moore* module.

1.4 Verilog Implementation

The implementation of the *vending_25c_moore* module was achieved using behavioral-level modeling in Verilog. Figure 1 shows the full code for this program. As shown in the figure, the port list of the program matches the expected inputs and outputs. Due to being used as outputs for behavioral-level modeling, *P*, *C*, *state* and, *next_state* are registers. For the functionality description, the module utilizes two procedural blocks. The first is for sequential logic, while the second is for the combinational logic. The sequential block uses a *posedge clock* to update the *state* register and implements the active-low synchronous reset to initialize the machine to 3'b000 (S0). As shown, within the combinational block, a case statement evaluates the state to assign the Moore outputs, *P* and *C* and calculate the *next_state* based on the *DQ* input vector. To prevent the creation of unintended latches, a default case is included in the case statement and else conditions are used in every if statement. This ensures that every possible input combination is accounted for, forcing the *next_state* to a known value in all scenarios.

```

vending_25c_moore(
    input clock,
    input reset_n,
    input [1:0] DQ,
    output reg P,
    output reg C
);

reg [2:0] state, next_state;
always @ (posedge clock) begin
    if (!reset_n) state <= 3'b000;
    else state <= next_state;
end

always @ (*) begin
    case (state)
        3'b000: begin
            P = 0; C = 0;
            if (DQ == 2'b10) next_state = 3'b001;
            else if (DQ == 2'b01) next_state = 3'b011;
            else next_state = 3'b000;
        end
        3'b001: begin
            P = 0; C = 0;
            if (DQ == 2'b10) next_state = 3'b010;
            else if (DQ == 2'b01) next_state = 3'b100;
            else next_state = 3'b001;
        end
        3'b010: begin
            P = 0; C = 0;
            if (DQ == 2'b10) next_state = 3'b100;
            else if (DQ == 2'b01) next_state = 3'b100;
            else next_state = 3'b010;
        end
        3'b011: begin
            P = 1; C = 0;
            next_state = 3'b000;
        end
        3'b100: begin
            P = 1; C = 1;
            next_state = 3'b000;
        end
        default: begin
            P = 0; C = 0;
            next_state = 3'b000;
        end
    endcase
end
endmodule

```

Figure 2: Source code for the main module, *vending_25c_moore*

2 Simulation, Results and Analysis

To verify the correctness of the *vending_25c_moore* module, two distinct simulation scenarios were executed. These scenarios confirm that the FSM transitions correctly under both ideal and non-ideal input conditions.

2.1 Scenario A: Ideal Input Sequence

The first simulation represents the ideal scenario using a sequence of two dimes followed by a quarter ($DQ = 10, 10, 01$). This specific sequence was chosen because it tests the FSM's ability to process both types of coin inputs ($DQ = 10$ and $DQ = 01$) within a single transaction. Furthermore, this combination results in a total of 45 cents, which allows for the verification of the overflow state (S4) where both the product (P) and change (C) outputs must be asserted simultaneously.

2.2 Scenario B: Non-Ideal Input Sequence

The second simulation verifies the FSM's ability to maintain its state during idle and invalid input conditions across multiple states. The chosen sequence consists of $DQ = 00, 11, 10, 00, 11, 10, 00$,

11, and finally 10. As seen in the simulation sequence, a "no coin" input ($DQ = 00$) and an invalid "quarter and dime" input ($DQ = 11$) are applied before each valid dime insertion ($DQ = 10$). This pattern specifically tests the non-ideal inputs of the FSM at critical states where an input is supposed to occur (S0, S1, and S2).

2.3 Testbench Programming

In order to verify the correctness of the *vending_25c_moore* module, two distinct simulation scenarios were executed. Each scenario was implemented using an independent testbench to ensure isolation of the test cases and to simplify the verification process. The first testbench, *vending_25c_moore_tb_ideal*, executes Scenario A to simulate the ideal input sequence. As shown in Figure 3, the testbench defines the required registers and wires to create an instantiation of the *vending_25c_moore* module. A clock with a 10ns period is generated using an *always* block that toggles every 5ns. The simulation drives *reset_n* low for 10ns to initialize the FSM to state S0. Following the reset, the testbench applies a sequence of two dimes ($DQ = 10$) and one quarter ($DQ = 01$) at 10ns intervals. In addition to generating waveforms, the testbench uses *\$display* tasks to output the signal values to the TCL Console in binary format. As shown in Figure 3, these values are displayed 10ns after each input change. This delay ensures that the *state* and outputs are recorded after the positive edge of the clock has triggered the next state update.

```
module vending_25c_moore_tb_ideal();
    reg clock;
    reg reset_n;
    reg [1:0] DQ;
    wire P;
    wire C;
    vending_25c_moore ul(clock, reset_n, DQ, P, C);
    initial clock=0;
    always #5 begin clock=~clock; end
    initial begin
        reset_n=1'b0;
        #10 reset_n=1'b1;
        DQ=2'b10;
        $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        #10
        DQ=2'b10;
        $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        #10
        DQ=2'b01;
        $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        #10
        $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        $finish;
    end
endmodule
```

Figure 3: Source code of the First Testbench, Scenario A.

The second testbench, *vending_25c_moore_tb_non_ideal*, executes Scenario B to verify the FSM's behavior under non-ideal input conditions. This simulation follows the inputs for *DQ* given for Scenario B. As shown in Figure 4 and similar to previous testbench, the testbench initiates by resetting the system to state S0 and has a 10ns period clock running with the program. Also similar to the first scenario, signal values are logged to the TCL Console 10ns after each input transition to ensure the captured data reflects the state update triggered by the rising clock edge.

```

module vending_25c_moore_tb();
    reg clock;
    reg reset_n;
    reg [1:0] DQ;
    wire P;
    wire C;
    vending_25c_moore ul(clock, reset_n, DQ, P, C);
    initial clock = 0;
    always #5 begin clock = ~clock; end
    initial begin
        reset_n = 1'b0;
        DQ = 2'b00;
        #10 reset_n = 1'b1;
        DQ = 2'b00;
        #10 $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        DQ = 2'b11;
        #10 $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        DQ = 2'b10;
        #10 $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        DQ = 2'b00;
        #10 $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        DQ = 2'b11;
        #10 $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        DQ = 2'b10;
        #10 $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        DQ = 2'b11;
        #10 $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        DQ = 2'b10;
        #10 $display("reset_n=%b DQ=%b | state=%b | P=%b C=%b", reset_n, DQ, ul.state, P, C);
        #10 $finish;
    end
end
endmodule

```

Figure 4: Source code of the Second Testbench, Scenario B.

2.4 Simulation Results

The outputs of the first testbench of the main module, executing scenario A, are shown through Figures 5 and 6. Figure 5 shows the waveform output from AMD Vivado of the first testbench and Figure 6 shows the output of the first testbench through the TCL Console.

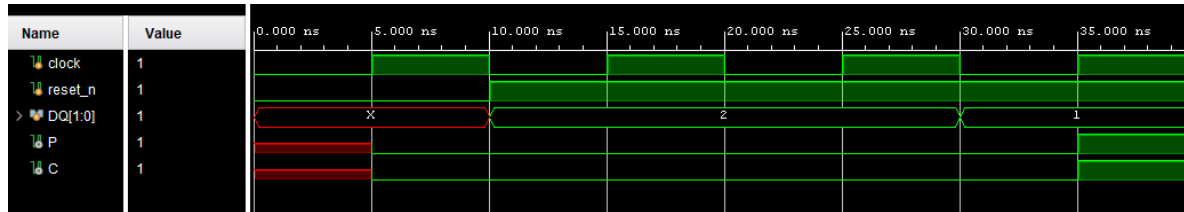


Figure 5: Waveform Output of the First Testbench, Scenario A

```

reset_n=1 DQ=10 | state=000 | P=0 C=0
reset_n=1 DQ=10 | state=001 | P=0 C=0
reset_n=1 DQ=01 | state=010 | P=0 C=0
reset_n=1 DQ=01 | state=100 | P=1 C=1

```

Figure 6: TCL Console Output of the First Testbench, Scenario A

The outputs of the second testbench of the main module, executing scenario B, are shown through Figures 7 and 8. Figure 7 shows the waveform output from AMD Vivado of the second testbench and Figure 8 shows the output of the second testbench through the TCL Console.

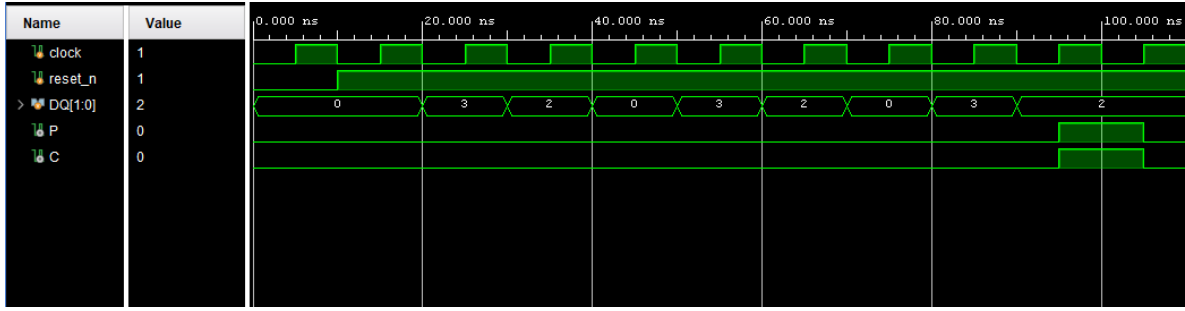


Figure 7: Waveform Output of the Second Testbench, Scenario B

```

reset_n=1 DQ=00 | state=000 | P=0 C=0
reset_n=1 DQ=11 | state=000 | P=0 C=0
reset_n=1 DQ=10 | state=001 | P=0 C=0
reset_n=1 DQ=00 | state=001 | P=0 C=0
reset_n=1 DQ=11 | state=001 | P=0 C=0
reset_n=1 DQ=10 | state=010 | P=0 C=0
reset_n=1 DQ=00 | state=010 | P=0 C=0
reset_n=1 DQ=11 | state=010 | P=0 C=0
reset_n=1 DQ=10 | state=100 | P=1 C=1

```

Figure 8: TCL Console Output of the Second Testbench, Scenario B

The simulation has been shown to be successful because the output waveforms and TCL binary logs perfectly match the expected state transition table for the Moore FSM.

These figures provide two different views of the outputs of the respective simulations. Figure 5 and 7, the waveforms, show the timing and transitions that take place during the simulation. It shows how the outputs respond to changes in DQ and when they are updated. Furthermore, the timing of the outputs confirm that this is a Moore state machine, as they are updated in accordance with the clock. This is unlike a Mealy state machine, where the outputs are dependent on both the current state and the current inputs, causing them to change asynchronously as soon as DQ changes, regardless of the clock edge.

The waveform for Scenario A, Figure 5, also confirms that the FSM transitions from S_0 to S_1 (10¢) and then to S_2 (20¢) when it receives consecutive $DQ = 10$ inputs. When the quarter ($DQ = 01$) is detected at S_2 , the FSM transitions to the overflow state S_4 (45¢ total). This confirms that the state register is updating correctly on each clock edge and that the outputs respond properly once the 25¢ threshold has been surpassed.

The waveform for Scenario B, Figure 7, further validates the design by forcing the machine to process idle (00) and invalid (11) bit combinations. The successful simulation confirms that the combinational next-state logic correctly implements the "else" branches and "default" cases, preventing unintended transitions. The results show that the machine holds its current state until a valid $DQ = 10$ signal is sampled at the clock edge. This stability also proves that the FSM is able to handle against non-ideal inputs and is free of unintended latches.

Figure 9 provides an expanded view of the output waveform between 0 ns and 12 ns. The figure shows the value of the DQ , being 2'bXX. This is also shown on Figure 5, though not as clear, with the red bar at 0 ns for DQ . As shown in Figure 9, DQ changes to a known value at 10 ns, where it is first defined in the testbench. The reason why DQ is defined at 10ns is because a reset needs to occur before the state machine can take in the first input for scenario A, $DQ = 11$ and the period of the clock in the testbench is 10 ns.

Figure 9 also shows the value of outputs P and C , being 1'bX from 0 to 5 ns. The outputs P and C remain unknown until 5 ns because the design utilizes a synchronous reset, as per project specifications. Consequently, the current state of the program only becomes defined at the first rising edge of the clock signal, which occurs at the 5 ns mark. Since the states provide values to the register

outputs P and C and there is no state defined, the outputs P and C are unknown for the first 5 ns. As the reset needs to be synchronous for this design, this initial delay is an inherent characteristic of synchronous reset logic rather than a functional error. From this analysis, it can be concluded that the initial unknown state of DQ, P , and C are expected behaviors rather than as a result from a functional error in the design.

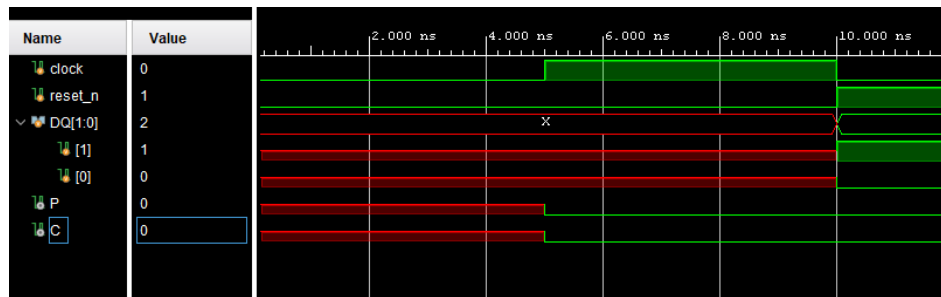


Figure 9: Enhanced Waveform of the first testbench from 0 ns to 12 ns

A potential limitation in the verification process is that only a small number of test cases was utilized. For the 2-bit input, DQ , there are $2^2 = 4$ possible combinations. When including the 1-bit $reset_n$ input, there are $2^3 = 8$ total possible input values per clock cycle. While the simulation scenarios cover the primary operational modes, they represent only a fraction of the possible input sequences over time. In addition to this, because the module is a sequential machine containing a state register, the output depends not only on the current DQ values but also on the *state* stored from previous cycles. This sequential nature means the total number of possible state-transition sequences grows exponentially over multiple clock periods. However, it is impractical to simulate every possible sequence due to the time required for manual verification and documentation. This is why representative test cases, specifically the ideal and non-ideal input sequences, were selected to confirm the success of the FDM under a variety of conditions.

As the outputs from the module matched the expected results, the Verilog program and its logical structure are deemed successful.

3 Conclusions

The correct implementation and verification of a vending machine Moore finite state machine (FSM) was achieved in this project. This was accomplished through behavioral modeling to define the state transitions and Moore outputs. A successful interpretation of a state machine diagram was demonstrated, confirming that the FSM accurately tracks total value and triggers the product and change outputs as expected. While the design was functionally successful, an area for future improvement involves timing and resource optimization. Since the test cases focused solely on functional correctness, separate timing analysis could be conducted to determine how the state register performance impacts emulated hardware of the system. Future design updates could incorporate these efficiency metrics when selecting test cases. Nonetheless, this project demonstrated the capabilities of Verilog to implement and verify sequential logic through behavioral programming, creating a modular structure that can be replicated for future FSM designs. Rather than requiring physical components like logic gates, wires, and chips, or expensive hardware like an FPGA, the design was fully verified digitally with minimal resources used.

This report was compiled using L^AT_EX .